

Lab 4: VibeCoding - LLM Chat History

Student SRN: PES1UG24CS384 | Scenario 15: Connect Four vs AI

Claude Chat Share Link: <https://claude.ai/share/56914cc2-0791-40fb-b8c1-a1e3acbea036>

PROMPT 1 (Task 1: Complete Win Detection)

User Prompt:

In board.py, update the winner method to detect 4-in-a-row in all four directions: horizontal (0, 1), vertical (1, 0), diagonal down-right (1, 1), and diagonal down-left (1, -1). Ensure shorter sequences do not count as wins.

LLM Response / Implemented Code (board.py):

```
def winner(self, token):
    # directions: horizontal, vertical, diagonal-down-right, diagonal-down-left
    directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
    for r in range(ROWS):
        for c in range(COLS):
            if self.grid[r][c] != token:
                continue
            for dr, dc in directions:
                if all(
                    0 <= r + i * dr < ROWS
                    and 0 <= c + i * dc < COLS
                    and self.grid[r + i * dr][c + i * dc] == token
                    for i in range(4)
                ):
                    return True
    return False
```

PROMPT 2 (Task 2: Complete Game Termination & Validation)

User Prompt:

In game.py, ensure proper input validation for column numbers (1 to 7), clean error messages when a column is full or invalid without crashing, and ensure winning moves and draws terminate the game cleanly.

LLM Response / Implemented Code (game.py):

```
while True:
    self.board.print()
    if self.turn == 'X':
        try:
            raw = input('Column (1-7), or q: ').strip().lower()
```

```

except (EOFError, KeyboardInterrupt):
    print('\nGoodbye.')
    return
if raw == 'q':
    print('Game quit.')
    return
try:
    col = int(raw) - 1
except ValueError:
    print('Please enter a number from 1 to 7, or q to quit.')
    continue
if not 0 <= col < COLS:
    print('Column must be between 1 and 7.')
    continue
else:
    col = self.ai.choose_column(self.board)
    if col is None or not 0 <= col < COLS:
        print('AI returned an invalid column; ending game.')
        return

if self.board.drop(col, self.turn) is None:
    if self.turn == 'X':
        print(f'Column {col + 1} is full. Choose another.')
        continue
    print('AI chose a full column; ending game.')
    return

if self.board.winner(self.turn):
    self.board.print()
    print(self.turn, 'wins!')
    return
if self.board.full():
    self.board.print()
    print('Draw.')
    return
self.turn = 'O' if self.turn == 'X' else 'X'

```

PROMPT 3 (Task 3: Improve the AI Decision Making)

User Prompt:

In ai.py, upgrade the AI decision-making: first check if the AI has an immediate winning move and take it; second check if the opponent has an immediate winning move on their next turn and block it; otherwise choose safely from the remaining legal columns. Handle full

columns safely.

LLM Response / Implemented Code (ai.py):

```
import copy
import random
COLS = 7

class AI:
    def choose_column(self, board, me='O', opponent='X'):
        legal = [c for c in range(COLS) if board.grid[0][c] == '.']
        if not legal: return None
        # 1. Take an immediate win.
        for col in legal:
            if self._wins_if_played(board, col, me):
                return col
        # 2. Block the opponent's immediate win.
        for col in legal:
            if self._wins_if_played(board, col, opponent):
                return col
        # 3. Choose safely: avoid moves handing opponent a win above & prefer center.
        safe = [c for c in legal if not self._gives_win_above(board, c, me, opponent)]
        candidates = safe or legal
        center = COLS // 2
        best = min(abs(c - center) for c in candidates)
        return random.choice([c for c in candidates if abs(c - center) == best])

    @staticmethod
    def _wins_if_played(board, col, token):
        trial = copy.deepcopy(board)
        if trial.drop(col, token) is None: return False
        return trial.winner(token)

    @staticmethod
    def _gives_win_above(board, col, me, opponent):
        trial = copy.deepcopy(board)
        if trial.drop(col, me) is None: return False
        if trial.grid[0][col] != '.': return False
        return trial.drop(col, opponent) is not None and trial.winner(opponent)
```

PROMPT 4 (Task 4: Move-Level Feedback)

User Prompt:

In game.py, add concise move-level feedback that prints which column the player or AI

successfully placed their disc into, appearing exactly once per actual move.

LLM Response / Implemented Code (game.py):

```
if self.board.drop(col, self.turn) is None:
    if self.turn == 'X':
        print(f'Column {col + 1} is full. Choose another.')
        continue
    print('AI chose a full column; ending game.')
    return

who = 'You' if self.turn == 'X' else 'AI'
print(f'{who} placed a disc in column {col + 1}.')
```

Feedback Mechanism Explanation:

- Only after success: placed after failed-drop branch so errors don't trigger messages.
- Before win/draw checks: announces the winning move before showing the terminal result.
- Prints col + 1 to correspond directly with 1-7 user input.

Example Console Output:

```
You placed a disc in column 4.
AI placed a disc in column 4.
```